

딥러닝 영상처리- Lecture Week12-1

yolo predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg' save_txt=True

bus.txt

```
5 0.493844 0.443415 0.978224 0.462082
0 0.914091 0.589604 0.171349 0.448036
0 0.176979 0.603593 0.236909 0.46725
0 0.350325 0.587558 0.149887 0.418284
0 0.0425353 0.661309 0.085017 0.292862
```

<class_id> <center_x> <center_y> <width> <height>

```
`absolute_center_x = center_x * image_width`
```

```
`absolute_center_y = center_y * image_height`
```

```
`absolute_width = width * image_width`
```

```
`absolute_height = height * image_height`
```

```

import cv2

image = cv2.imread('bus.jpg')

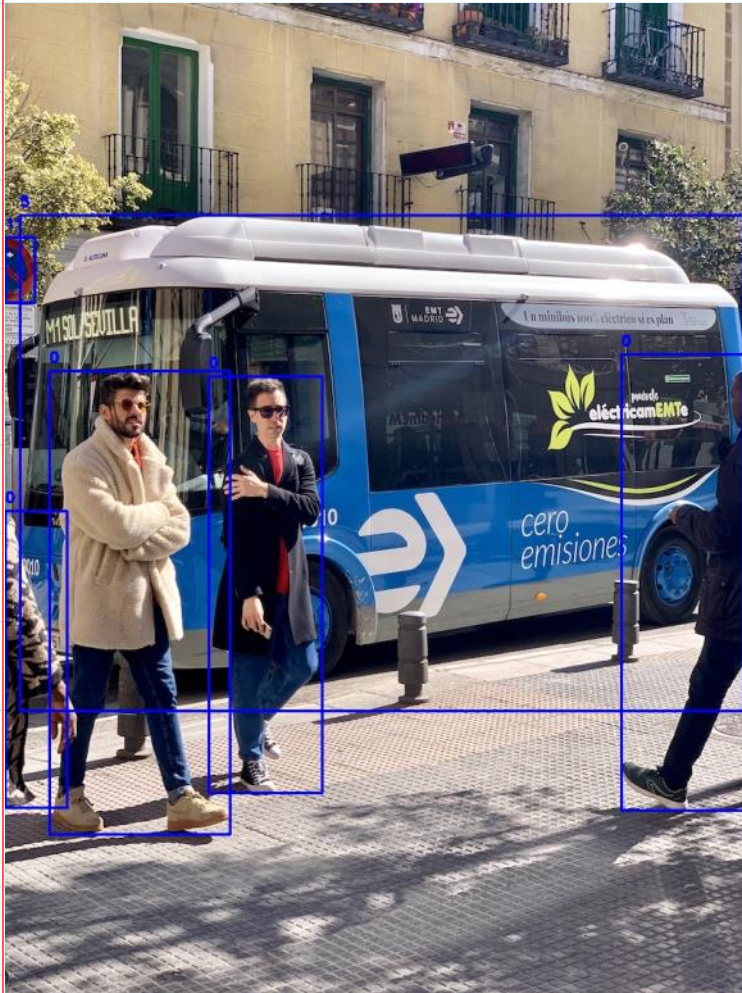
boxes = [
    (5, 409.40, 499.50, 785.25, 538.81),
    (0, 146.62, 651.29, 195.98, 503.24),
    (0, 739.06, 628.99, 139.59, 495.41),
    (0, 283.05, 631.59, 123.18, 451.60),
    (11, 16.18, 289.85, 32.22, 70.39),
    (0, 33.55, 712.47, 67.14, 322.93)
]

for box in boxes:
    class_id, center_x, center_y, width, height = box
    top_left_x = int(center_x - width / 2)
    top_left_y = int(center_y - height / 2)
    bottom_right_x = int(center_x + width / 2)
    bottom_right_y = int(center_y + height / 2)

    # 네모 박스 그리기 (파란색, 두께 2)
    cv2.rectangle(image, (top_left_x, top_left_y), (bottom_right_x, bottom_right_y), (255, 0, 0), 2)
    # 클래스 ID 텍스트 추가 (빨간색, 두께 2)
    cv2.putText(image, str(class_id), (top_left_x, top_left_y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

# 이미지 저장 또는 표시
cv2.imwrite('bus draw boxes.jpg', image)

```



yolo detect train data=coco8.yaml model=yolo11n.yaml epochs=100 imgsz=640

<https://docs.ultralytics.com/modes/train/>

실시간 비디오 객체 탐지 및 결과 저장

```
import cv2
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

video_path = "./sample.mp4"
cap = cv2.VideoCapture(video_path)

fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter('./output.mp4', fourcc,
                      cap.get(cv2.CAP_PROP_FPS),
                      (int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
                       int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))))

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model.predict(source=frame, conf=0.3, verbose=False)

    annotated_frame = results[0].plot()

    out.write(annotated_frame)

    cv2.imshow('YOLOv11 Video Inference', annotated_frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
out.release()
cv2.destroyAllWindows()
```

Fps 측정 및 Latency 결과 추가

```
import cv2
import time
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

cap = cv2.VideoCapture("sample.mp4")
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter("output_with_cumulative_delay.mp4", fourcc,
                      cap.get(cv2.CAP_PROP_FPS),
                      (int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
                       int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))))

cumulative_delay = 0.0

while cap.isOpened():
    loop_start = time.time()

    ret, frame = cap.read()
    if not ret:
        break

    infer_start = time.time()
    results = model.predict(source=frame, conf=0.3, verbose=False)
    infer_end = time.time()

    annotated_frame = results[0].plot()

    infer_time = infer_end - infer_start
    loop_time = time.time() - loop_start
    delay_time = loop_time - infer_time
    cumulative_delay += delay_time

    fps = 1.0 / loop_time
    cv2.putText(annotated_frame, f"FPS: {fps:.2f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
    cv2.putText(annotated_frame, f"Delay: {delay_time:.3f}s", (10, 65),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 100, 255), 2)
    cv2.putText(annotated_frame, f"Cumulative Delay: {cumulative_delay:.2f}s", (10, 100),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 100, 100), 2)

    out.write(annotated_frame)
    cv2.imshow("YOLOv11 Inference with Cumulative Delay", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
out.release()
cv2.destroyAllWindows()
```

사람 count 기능 추가

```
import cv2
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

video_path = "./sample.mp4"
cap = cv2.VideoCapture(video_path)
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter('./output.mp4', fourcc,
                      cap.get(cv2.CAP_PROP_FPS),
                      (int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
                       int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))))

person_count = 0

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model.predict(source=frame, conf=0.3, verbose=False)

    class_ids = results[0].boxes.cls.cpu().numpy().astype(int)

    if 0 in class_ids:
        person_count += 1

    annotated_frame = results[0].plot()

    cv2.putText(annotated_frame, f"Person count: {person_count}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2)

    out.write(annotated_frame)
    cv2.imshow('YOLOv11 Video Inference', annotated_frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
out.release()
cv2.destroyAllWindows()
```

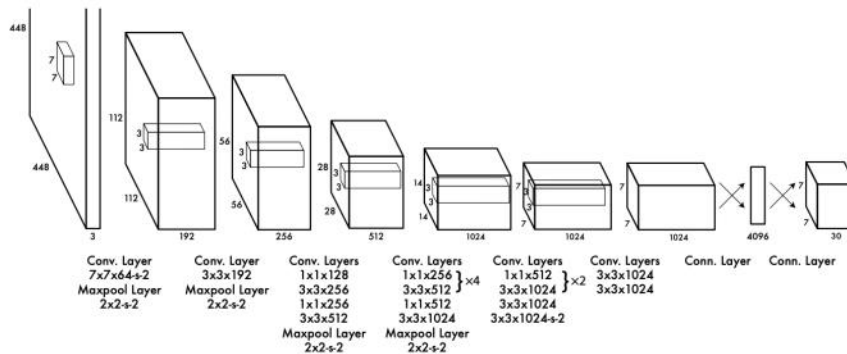
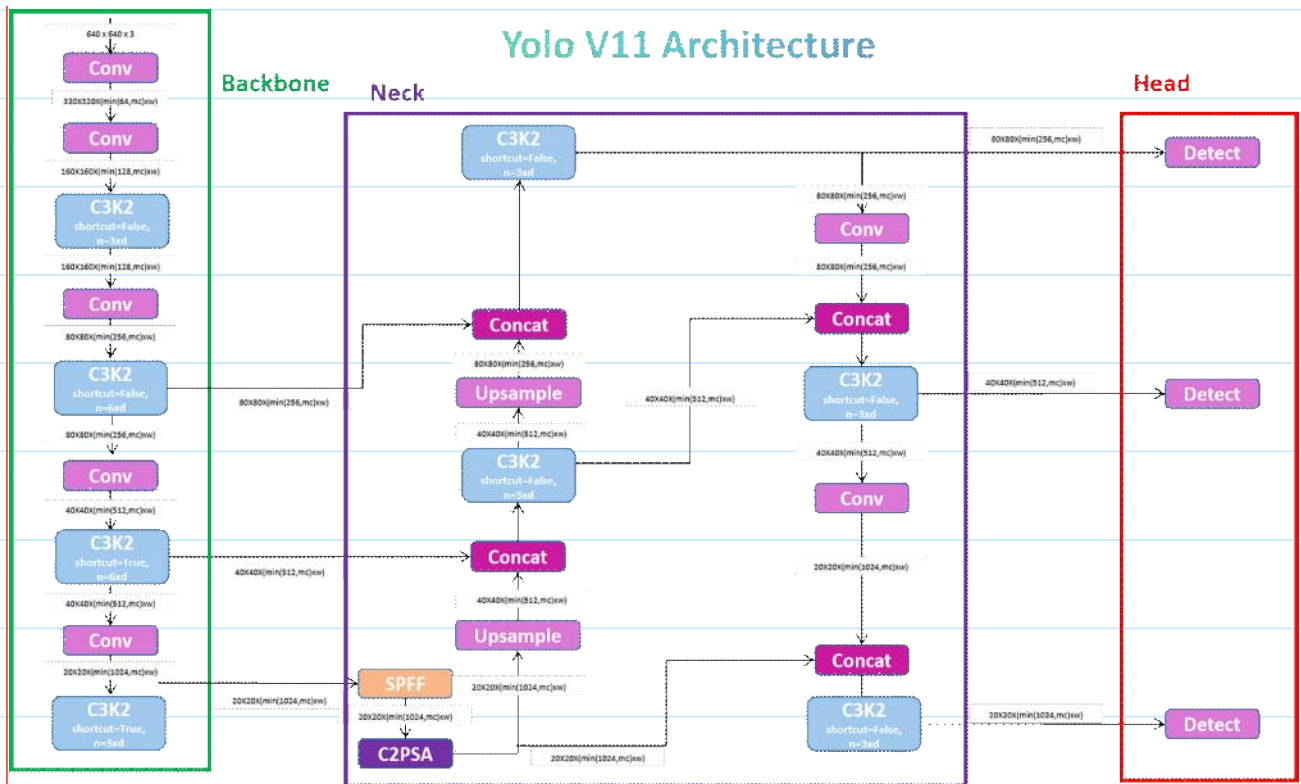
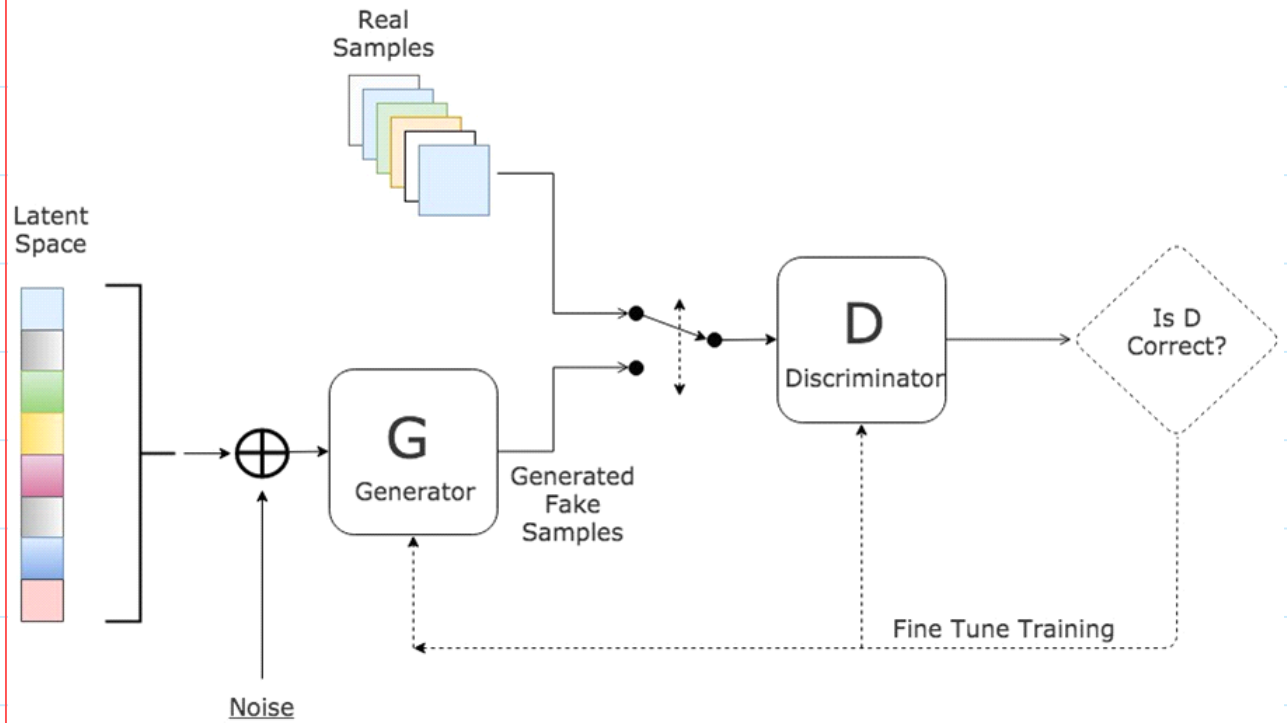


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Generative Adversarial Network



GAN 모델 학습 과정

1. Generator 학습

- 1) 랜덤 노이즈 벡터를 입력 받아 가짜 데이터 생성
- 2) 가짜 데이터를 Discriminator에 입력하여, Discriminator의 출력 확인
- 3) Discriminator가 가짜 데이터를 진짜로 분류하도록 Generator의 파라미터를 업데이트

2. Discriminator 학습

- 1) 진짜 데이터와 Generator가 생성한 가짜 데이터를 모두 입력 받아, 이를 진짜와 가짜로 분류
- 2) 진짜 데이터를 진짜로, 가짜 데이터를 가짜로 분류하도록 Discriminator의 파라미터를 업데이트



.